Video Intelligence

Captioning, tracking and segmentation of video, one web app. Notes for tomorrow's discussion.

■ The 30-second pitch

- I extended my CLIP + BLIP video captioning project into a full video-understanding app.
- Three features, each its own page: **captioning**, **tracking**, **segmentation**.
- A fourth page merges all three into one annotated video with one dashboard.
- The web app only calls my original pipeline files. I did not rewrite them.
- Voice-over (TTS) stays in the repo but is not part of the vision work.

■ What I built on top

- **Tracking:** YOLOv8 + ByteTrack / BoT-SORT, unique counts, dwell time, line crossings, heatmap, trajectories.
- **Segmentation:** YOLOv8-seg masks, per-class area statistics over time.
- **Evaluation:** proxy metrics for captioning, MOT export for IDF1 / MOTA / HOTA, mask IoU / Dice.
- **Web app:** Flask, background jobs with a progress bar, Chart.js dashboards.

How the pieces connect

Page	Input	Models	Output
Captioning	video	CLIP ViT-B/32, BLIP base, Groq LLM	EN + AR captioned videos, caption metrics
Tracking	video	YOLOv8 + ByteTrack (or BoT-SORT)	boxes + IDs + trails, dashboard, MOT file
Segmentation	video	YOLOv8-seg	masks, class area dashboard
Full analysis	video	YOLOv8-seg + tracker + BLIP captions	one video + one dashboard

Where the code is (webapp/features)

Folder	Steps, in order
captioning	your root files 1_Frame_Extraction, 2_Keyframe_Selection_CLIP, 3_Captioning_BLIP, 4_Collect_Captions, 6_burn_captions; pipeline.py wires them; 5_evaluation.py
tracking	1_detection, 2_tracking, 3_analytics, 4_render, 5_evaluation, pipeline.py
segmentation	1_segmentation_model, 2_masks, 3_render, 4_statistics, 5_evaluation, pipeline.py
full_analysis	pipeline.py only: re-uses the steps above

Pipeline 1 · Video Captioning (CLIP + BLIP)

Video in, English and Arabic captioned videos out. Keyframes first, captions second.

PIPELINE

1 Video input
mp4 / mov
OpenCV reads frames

→

2 Frame sampling
2 frames / second
(fps_sample = 2)

→

3 CLIP keyframes
Embed each frame
(512-d, L2-normalised)
Keep if cosine sim to
last kept frame < 0.96

→

4 BLIP captions
One caption per keyframe
Beam search, 5 beams
max 30 new tokens

→

5 LLM narrative
Denoise duplicates, then
Groq LLM merges into a
story in EN / AR

→

6 Burn captions
Arabic reshaped + bidi
Captions drawn on video
(moviepy)

→

7 Output
captioned_en.mp4
captioned_ar.mp4
+ metrics JSON

MODELS: WHAT THEY DO, WHY, AND HOW THEY ARE BUILT

CLIP ViT-B/32 (keyframe selection)

Why: raw pixel differences fire on lighting and noise. CLIP embeddings compare meaning, so a frame is kept only when the scene really changes.

Architecture: two encoders, one shared space
- Image: ViT-B/32, 32×32 patches, 12 layers
- Text: 12-layer Transformer
- Both project to a 512-d vector

Loss: symmetric InfoNCE (contrastive).
- Matching image-text pairs pulled together,
- all other pairs in the batch pushed apart.

Trained on 400M image-text pairs.

BLIP base (caption generation)

Why: strong captions from a pretrained model, no training needed. Fits our use: short, factual descriptions of each keyframe.

Architecture: Multimodal mixture of Encoder-Decoder
- ViT image encoder
- Text decoder with cross-attention to the image
- (we use the image-grounded decoder)

Losses in pretraining: ITC (contrastive) + ITM
- (match / no match) + LM (next-word prediction).

CapFilt: a captioner writes synthetic captions,
- a filter removes noisy ones, then retrain.

Evaluation

Without reference captions (what we run):
- Compression ratio = frames / keyframes
- CLIPScore = 2.5 * max(cos(img, caption), 0)
- Redundancy = word overlap of neighbour captions
- Unique captions %, per-stage runtime

With human reference captions (if available):
- BLEU-4, METEOR, ROUGE-L
- CIDEr (main COCO caption metric)
- SPICE (objects, attributes, relations)

CLIP stage itself: report keyframe recall against hand-marked scene changes.

Captioning: say it in one breath

What I say

- Sample 2 frames per second, embed each with CLIP.
- Keep a frame only if its cosine similarity to the last kept frame is below 0.96.
- BLIP captions each keyframe (beam search, 5 beams).
- An LLM merges the captions into a short story in English and Arabic.
- Captions are burned onto the video.
- **Why CLIP for selection:** it compares meaning, not pixels, so lighting changes do not trigger a keyframe.
- **Why BLIP:** strong pretrained captioner, no training needed.

If they push

- **CLIP loss:** symmetric InfoNCE. Matching pairs pulled together, other pairs in the batch pushed apart.
- **BLIP losses:** ITC + ITM + LM. CapFilt cleans noisy web captions.
- **Threshold trade-off:** higher = more keyframes, more detail, slower.
- **Metric with no references:** CLIPScore = 2.5 * max(cos(image, caption), 0).
- **Metric with references:** CIDEr (main), BLEU-4, METEOR, ROUGE-L, SPICE.
- **Compression ratio** shows how much redundancy CLIP removed.

Honest limits (say them before they ask)

- CLIPScore rates image-text agreement, not fluency or Arabic quality.
- Arabic is produced by the LLM from English captions, so it inherits BLIP's mistakes.
- BLIP base describes scenes, not identities or fine actions.
- I have no human reference captions, so CIDEr / BLEU are not reported.

Pipeline 2 · Object Tracking

Detect every object, keep one ID per object through the video, then analyse what happened.

PIPELINE

1 Video input
Read frame by frame
(streamed, low memory)

→

2 YOLOv8 detect
Boxes + class + score
conf >= 0.3
COCO: 80 classes

→

3 Predict
Kalman filter predicts
where each existing
track should be now

→

4 Associate
IoU cost matrix
Hungarian matching
ByteTrack: 2 passes
(high, then low score)

→

5 Update tracks
Matched: update state
Unmatched: kept alive for
a buffer, then dropped
New: start a new ID

→

6 Analytics
Unique count per class
Dwell time, centre-line
crossings, heatmap,
trajectories

→

7 Output
Annotated video
+ dashboard
+ MOT txt for scoring

MODELS: WHAT THEY DO, WHY, AND HOW THEY ARE BUILT

YOLOv8 (detector)

Why: fast single-stage detector, good accuracy,
sizes n/s/m/l let us trade speed for accuracy.

Architecture:
- Backbone: CSPDarknet with C2f blocks + SPPF
- Neck: PAN-FPN (top-down + bottom-up fusion)
- Head: decoupled (separate box / class branches),
anchor-free, 3 scales (strides 8, 16, 32)

Loss: CIoU + DFL (box), BCE (class).
Task-Aligned Assigner picks positive samples.
Detector metric: mAP@50 and mAP@50-95.

ByteTrack (default) / BoT-SORT

Why ByteTrack: keeps low-confidence boxes in a
second matching pass, so occluded objects keep
their ID. Very fast; no appearance network.

Building blocks: Kalman filter (motion model)
+ Hungarian algorithm (assignment)
+ IoU distance

Why BoT-SORT (option): adds camera-motion
compensation, so it holds up when the camera
moves. Can add ReID appearance features.
No tracker-specific loss: trackers are not trained.

Evaluation (needs ground truth)

Standard MOT metrics on MOT-format files:
- MOTA = 1 - (FN + FP + IDSW) / GT
detection-heavy
- IDF1 = identity-consistency over time
association-heavy
- HOTA = sqrt(DetA * AssA), balances both
- ID switches, fragmentations

Tools: motmetrics (MOTA, IDF1), TrackEval (HOTA).
Benchmarks: MOT17, MOT20.

No ground truth? The dashboard shows ID churn,
short-track % and mean track length as hints.

Tracking: say it in one breath

What I say

- YOLOv8 finds objects in every frame.
- A Kalman filter predicts where each known object should be.
- Hungarian matching on IoU links detections to tracks, so each object keeps one ID.
- **ByteTrack** also uses low-confidence boxes in a second pass, so partly hidden objects keep their ID.
- **BoT-SORT** adds camera-motion compensation for moving cameras.
- Analytics on top: unique count per class, time on screen, centre-line crossings, heatmap, trajectories.

If they push

- **SORT:** Kalman + Hungarian, no appearance. Fast, many ID switches.
- **DeepSORT:** adds a ReID appearance embedding. Fewer switches, slower.
- **ByteTrack:** SORT-style plus the low-score second pass.
- **MOTA:** detection-heavy (misses, false positives, ID switches).
- **IDF1:** how consistently an identity is kept over time.
- **HOTA:** sqrt(DetA * AssA), balances detection and association.
- Trackers have no loss: they are not trained. The detector is.

Honest limits (say them before they ask)

- Without ground truth I cannot report MOTA / IDF1 / HOTA. The app exports MOT files so they can be computed.
- Dashboard 'tracker health' (ID churn, short tracks) are hints, not accuracy.
- Counts are only as good as the detector: unseen objects are not counted.
- ByteTrack has no appearance model, so identities can swap after long occlusions.

Pipeline 3 · Instance Segmentation

A pixel mask for every object, not just a box. Optionally merged with tracking and captions.

PIPELINE

1 Video input
Read frame by frame

→

2 Backbone + neck
YOLOv8-seg:
C2f backbone, SPPF,
PAN-FPN features

→

3 Two outputs
Per object: box, class,
32 mask coefficients.
Per image: 32 prototype
masks (1/4 resolution)

→

4 Build masks
mask = sigmoid(
coefficients x prototypes)
then crop to the box

→

5 Render
Coloured transparent
mask + outline per object
(colour = object ID)

→

6 Statistics
Instances per class
Mean / max frame area
Area over time

→

7 Output
Segmented video
+ dashboard
(Full analysis adds IDs
and captions)

MODELS: WHAT THEY DO, WHY, AND HOW THEY ARE BUILT

YOLOv8-seg (instance segmentation)

Why: one pass, real-time, and it shares the same backbone as our tracker's detector. Mask R-CNN is two-stage and slower on video.

Architecture (YOLACT-style mask branch):
- Same backbone + neck as YOLOv8 detection
- Proto head: 32 prototype masks for the image
- Detection head also predicts 32 coefficients per object. Mask = coefficients x prototypes.

Loss: box (CIoU + DFL) + class (BCE) + mask (binary cross-entropy on the mask).
Trained on COCO instance masks.

Why not the alternatives

Mask R-CNN: accurate, but two-stage and slower.

SAM / SAM 2: segments anything and tracks masks, but is heavy and has no class labels by itself. A good upgrade path, not the real-time default.

Semantic segmentation (U-Net, DeepLab): labels each pixel by class but cannot separate two people standing together. We need instances so each object can also get a tracking ID.

Evaluation (needs ground truth)

Instance-level:
- Mask mAP@50 and mAP@50-95 (COCO style)

Per mask:
- Mask IoU = overlap / union
- Dice = 2 * overlap / (sum of areas)

Boundary quality (optional): boundary F-score

Ultralytics: model.val(data=...) reports mask mAP.
Our code: segmentation/5_evaluation.py (mask_iou, dice).

No ground truth? The dashboard shows confidence and area statistics per class. They describe the output but are not accuracy.

Segmentation: say it in one breath

What I say

- YOLOv8-seg is single-stage: one pass gives boxes, classes and masks.
- The network outputs 32 prototype masks for the image and 32 coefficients per object.
- **Mask = sigmoid(coefficients x prototypes)**, cropped to the box.
- Same backbone family as my tracker's detector, so both run fast together.
- Dashboard shows instances per class and the share of the frame each class covers over time.

If they push

- **Instance vs semantic:** instance separates two people standing together; semantic does not.
- **Why not Mask R-CNN:** two-stage, slower on video.
- **Why not SAM:** heavy, no class labels by itself. Good upgrade path.
- **Loss:** box (CIoU + DFL) + class (BCE) + mask (binary cross-entropy).
- **Metrics:** mask mAP@50 and 50-95, mask IoU, Dice.

Honest limits (say them before they ask)

- Pretrained on COCO's 80 classes. Unusual objects are not segmented.
- No ground-truth masks for my videos, so no mask mAP / IoU numbers yet.
- Masks are per frame. Flicker is possible; tracking IDs help but do not smooth masks.
- Mask resolution is limited by the prototype size (1/4 of the input).

Evaluation cheat sheet

Stage	Metric	In one line	Needs
CLIP keyframes	Compression ratio	frames sampled / keyframes kept	nothing
BLIP captions	CLIPScore	2.5 * max(cos(image, caption), 0)	nothing
BLIP captions	CIDEr, BLEU-4, METEOR, ROUGE-L, SPICE	overlap with human captions	reference captions
Detector	mAP@50, mAP@50-95	box quality across IoU thresholds	labelled boxes
Tracker	MOTA	1 - (FN + FP + ID switches) / GT	MOT ground truth
Tracker	IDF1	F1 of identity matches over time	MOT ground truth
Tracker	HOTA	sqrt(detection accuracy * association accuracy)	MOT ground truth
Segmentation	Mask mAP, mask IoU, Dice	overlap of predicted and true masks	labelled masks

How to get real tracking numbers

- Download a short sequence with ground truth (for example from MOT17), or hand-label 20-30 seconds of your own video.
- Run the app on it and download **tracks_mot.txt**.
- **MOTA, IDF1, ID switches:** webapp/features/tracking/5_evaluation.py, evaluate_mot(gt_path, pred_path) (needs *pip install motmetrics*).
- **HOTA:** official TrackEval on the same two files.

Numbers to fill in after your runs

Item	Your value	Where to read it
Frames sampled / keyframes / compression ratio		Captioning page, metrics row
Mean CLIPScore		Captioning page
Processing speed (fps), model size, hardware		Tracking / Full analysis page
Unique objects, ID churn, short tracks %		Tracking dashboard
IDF1 / MOTA / HOTA on a labelled clip (optional)		motmetrics + TrackEval
Mask mAP or IoU on a labelled clip (optional)		model.val() or segmentation/5_evaluation.py

Likely questions, short answers

Question	Answer
Why track at all?	Counting unique objects needs identity across frames; detection alone counts the same person many times.
Why ByteTrack?	Strong accuracy with no extra network. Its second pass recovers low-confidence, occluded objects.
When BoT-SORT?	Moving camera, or when appearance features are needed.
What if IDs switch a lot?	Raise confidence, increase track buffer, use BoT-SORT with ReID, or use a larger detector.
How do you know it works?	Honest answer: proxy metrics on my videos, plus the MOT export to compute IDF1 / MOTA / HOTA against labels.
Biggest weakness?	Everything is pretrained on COCO. No custom training or ground-truth evaluation yet.
Next step?	Label a short clip, report IDF1 / HOTA; fine-tune the detector on my domain; try SAM 2 for mask tracking.